QUANSER
INNOVATE·EDUCATE

Lab Procedure
LCD Display

Introduction

Ensure the following:

1. You have reviewed the Application Guide – LCD Display
2. Make sure the **Mechatronic Sensors Trainer** is out of its case; it is powered using the provided USB cable connected to the PC.
    a. The LED next to the USB C port will stay white after the touch screen starts loading.
    b. The load screen with the Quanser Logo should disappear after a few seconds and you should see some evenly spaced crosses on a black background on the touch screen.

Exploring LCD Display

1. Launch Visual Studio Code or your preferred Python IDE. If using Visual Studio Code, click on **File** > **Open Folder**..., and browse to the directory containing the python files and lab procedure for the LCD Input lab (…/sensors_trainer/1_fundamentals /lcd_input/hardware/python). Open this directory.
2. Open lcd_input.py. In the Experiment constants region, make sure **`singleFinger`** is set to **`True`**.
3. Run lcd_input.py using the  button or the terminal. This will print the location of touch input to the terminal every half second while there is a finger on the screen.
4. Touch the screen and move your finger around the screen. For this example, it will only read one finger. Try to get to all edges, how many columns and rows does the display have? Where is (o,o) and where is (maxCol, maxRow)?
5. Open the Display's Datasheet located in the LCD Input folder (…/sensors_trainer/1_fundamentals/ lcd_input). Read the first 3 pages. Does the number of pixels you observed match what you see in the datasheet?
6. Click back on the terminal in Visual Studio Code and stop the script using **`Ctrl+C`** and clicking enter when prompted (the script stops after 300 seconds automatically).
7. Run the same script again and test touching the screen with different objects or with different materials between your finger and the screen. What do you notice? Which materials work as a touch input? Which ones do not work? Under what conditions?
8. Stop the script.

9. In the Experiment constants region of the same file, update **singleFinger** to **False**. This will now recognize and print locations of your finger on the terminal for more than a single touch point.

10. Run the code again.

11. Put one finger on the display and see the values printed. What happens if you add a second finger? Note what happens to the **finger ID** and its **location** if you remove the first finger you placed on the screen? Note that the number of prints per line depends on the number of fingers reported by the display as **touch.num_fingers**.

12. Keep testing the touch inputs with more fingers. What is the maximum number of fingers you can read with this display? Does this match the datasheet?

13. Click back on the terminal in Visual Studio Code and stop the script using **Ctrl+C** and clicking enter when prompted (the script stops after 300 seconds automatically).

Plotting Touch Input

14. On the same folder, open lcd_draw_touch.py. At the top of the script, find **plotTouch** and make sure it is set to **False**.

15. Run your file using the  button or the terminal. This will place a small square at the location of your finger on the screen. It can handle up to 3 fingers and colors.

16. Play around with drawing on the screen. Note that at any point you can clear the screen by pressing Button 0 on your device. The code will run for 300 seconds as defined in the **simulationTime** variable; you can modify it if needed.

17. What happens when you move your fingers fast vs slow? Note that the touch panel samples at 120Hz.

18. What happens when you are drawing with two fingers and you remove the first one that touched the screen? If you were coloring something, would you want your colors to behave that way once one finger is removed? How do you think you could fix this?

19. The display image is updated in the function control_loop(). The current image is stored in the array **image**, declared as a 3-dimensional array. What do each of the dimensions represent? In the following code, describe which elements are updated and in which order and explain why:

    ```
    image[finger1.r:finger1.r+squareSize,
    finger1.c:finger1.c+squareSize,:] = colorSquare1
    ```

    Why do we use **squareSize** here?

20. Stop your code.

21. At the top of the script, find **plotTouch** and set it to **True**.

22. Run your file using the  button or the terminal. This will plot your touch input on your computer.

23. Observe the plot and the touch screen as you draw horizontal lines across the middle, top, and bottom of the screen. Try drawing vertical lines going from top to bottom Explain what you observe.

24. Observe the **update_plot** function, and notice how the points **finger1**, **finger2**, and **finger3** are defined. Is the order of elements the same as when the **image** array gets updated? Why could the plots vary?

25. Modify **update_plot** so the plot directions match what you draw on the screen. You do not need the numbering of the axes to match.

26. Run your file again to make sure the scope looks correct. Write Hello on the display and save a screenshot of your plot.

27. Stop your code.

Writing to the Display

As observed in the previous sections of the lab, the touch area is also a screen we can write to. In this section, you will explore the different modes that you can use to draw on the screen.

28. On the same folder, open test_draw_modes.py. This will import 3 images in that folder and create the variable **white** (which creates a white image the size of the display) and display them using different draw modes from the lcd library.

29. In the same folder, open the images and view them on your computer.

30. Run the code and view the images on the lcd screen as they are displayed with different draw modes. Consider how each of the functions change the images. You may need to run the code multiple times. You can comment out examples, so they display one at a time or increase the sleep time between each step if it makes it easier to follow.

Note: the LCD update rate is at best around 26Hz, so when developing your code in the future, if you write to the screen faster than that, it might cause multiple updates to happen at the same time.

31. From what you see in these examples, how is the mask displayed differently in the **draw_image_mask** and **draw_image_blend** functions?

32. Based on what you saw in this file, what would be the difference between using **draw_image_mask** and **draw_image_blend** in the lcd_draw_touch.py file?

33. Stop, save and close your program.

34. Power OFF the Mechatronic Sensors Trainer by disconnecting its USB C cable, disconnect any external sensors and pack them along the base and the USB cables in the provided case.